Introduction to Linux OS

By Mohamed Gamal

Finding Files – find Command

What if you want to locate a file or directory? The first command you can use is the `find` command. It can be used to find files by name, size, permissions, owner, modification time, and more.

```
$ find [path] [expression]
```

Command	Usage	Example

`find path -name pattern`	Displays files/directories whose name matches pattern. This is case sensitive.	`$ sudo find / -name firefox`
		`$ find /usr/local -name *conf`
		`$ find . -name "*.sh"`

`find path -iname pattern`	Same as **-name** but ignores case.	`$ find . -iname "log*"` `$ find /var -iname apache`
`find path -regex pattern`	Search using regular expressions.	`$ find . -regex ".*\(txt|log\)$"`

`find path -type d|f`	Find only files or directories.	`$ find / -type d -name log`
		`$ find . -type f -name "pkc*"`
		`$ find . -type d -name firefox`

`find path -ls`	Performs an ls on each of the found files or directories.	`$ find . -ls` `$ find / -name "*.sh" -ls`

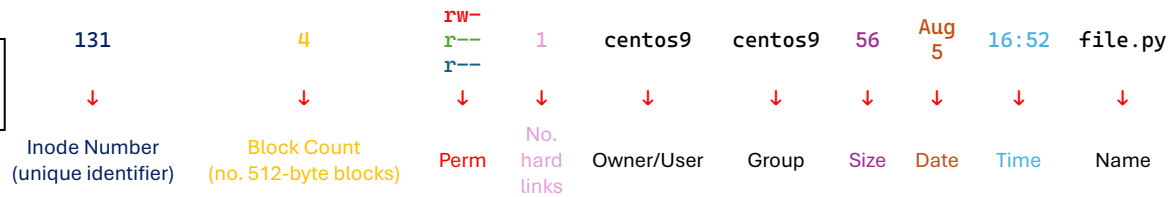
`find path -mtime [-| |+]days`
`-` `+`
`Last, Exactly, More than`
	mtime: modified time
	Finds files that are num_days old.
		`$ find . -mtime -7`
		`$ find . -mtime 5`
		`$ find . -mtime +30`
		`$ find . -mtime +10 -mtime -13`
		Files that are more than 10 days old, but less than 13 days old.

`find path -size [-| |+]size`	Finds files that are of size less/greater/equal to num.	`$ find . -size -5k`
		`$ find . -size 4G`
		`$ find . -size +3M`
		`$ find . -size +1M -mtime -3M`

`find path -user name`	Find files owned by a specific user.	`$ find /home -user MG`
`find path -newer file`	Finds files that are newer than file (i.e., modified after).	`$ find . -newer log.txt`

`find path -exec command {} \;`	Run command against all the files that are found.	`$ find . -type f -exec file {} \;`
		`$ find . -name "*.txt" -exec rm {} \;`
		`$ find . -name "*.log" -exec mv {} /var/logs/ \;`

Understanding the Output of -ls flag



Finding Files – locate Command

```
$ locate f*
```

Command	Usage	Example

`$ locate name`	Displays files with given name. This is case sensitive.	`$ locate firefox`
		`$ locate config.json`
		`$ locate log`

`$ locate -i name`	Case-insensitive search.	`$ locate -i FIREFOX`
`$ locate -r pattern`	Search using regular expressions.	`$ locate -r "\.txt$\|\.log$"`
`$ locate -c name`	Count the number of matching results.	`$ locate -c firefox`

find vs. Locate

Feature	`find` Command	`locate` Command

Search Method	Real-time traversal	Pre-built database (mlocate.db)
		`$ sudo cat /var/lib/mlocate/mlocate.db`
		`$ sudo updatedb`

Speed	Slower	Faster
Accuracy	Always up-to-date	May be outdated (unless `$ updatedb` ran)
Dependencies	None	Requires `mlocate` and `updatedb`
Customization	Highly customizable	Limited
Best For	Complex searches, real-time data	Quick name-based searches

Comparing Files

If you want to compare two files and display the differences you can use `diff`, `sdiff`, or `vimdiff`.

file1.txt	file2.txt
Hello World This is a test file. It contains some sample text. Let's compare these files using diff. The quick brown fox jumps over the lazy dog.	Hello Linux This is a test file. It contains some modified text. Let's compare these files using the diff command. The quick brown fox leaps over the lazy cat.

`$ diff file1 file2`

Example	Description
`$ diff file1.txt file2.txt`	Line-by-Line Comparison. **<** indicates content from file1.txt **>** indicates content from file2.txt **XcY** → Change (c) from line(s) X in file1.txt to line(s) Y in file2.txt.
`$ diff -y file1.txt file2.txt`	Side-by-Side Comparison. The | symbol means the line is different between the two files. Use `--suppress-common-lines flag to show only different lines.`
`$ diff -i file1.txt file2.txt`	Ignore case differences.
`$ diff -r dir1 dir2`	Recursively compares **all files** inside dir1 and dir2.

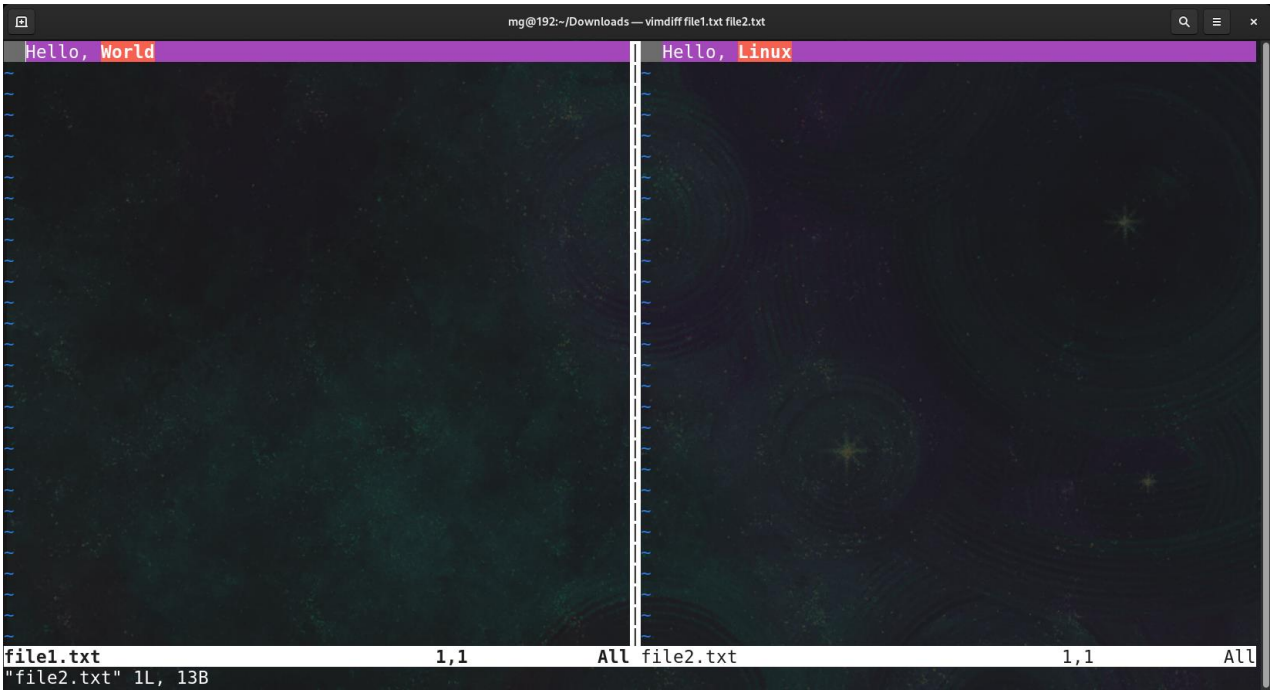
`$ sdiff file1 file2`

Example	Description
`$ sdiff file1.txt file2.txt`	Side-by-Side Comparison. Use either `--suppress-common-lines or -s flags to show only different lines.`
`$ sdiff -i file1.txt file2.txt`	Ignore case differences.
`$ sdiff -o merged.txt file1.txt file2.txt`	Automatically merge two files.

`$ vimdiff file1 file2`

Highlight the differences between two files in the vim editor

Action	Command
Switch windows	`Ctrl + w + w/⇔`
Next difference	`]c`
Previous difference	`[c`
Get change from right	`:diffget  or do`
Put change to right	`:diffput  or dp`
Recalculate differences	`:diffupdate`
Quit Vimdiff	`:q or :qa`



Sorting Files

In the simplest form the `sort` command sorts lines of text alphabetically.

file.txt

```
Hello World
This is a test file.
It contains some sample text.
It contains some sample text.
Let's sort this file using sort.
The quick brown fox jumps over the lazy dog.
```

data.csv

```
1, Mohamed, 5000, Egypt
2, John, 3500, USA
3, Mona, 4100, Germany
4, Suhaib, 2900, Pakistan
```

Command	Description
`$ sort file`	Sort text in file.
`$ sort -r file`	Sort in reverse order.
`$ sort -k F file`	Sort by key. The F following -k is the field/column number.
`$ sort -u file`	Sort text in file uniquely, removing duplicate lines.

```
$ sort file.txt
$ sort -u file.txt
$ sort -ru file.txt
$ sort -u -k 2 data.csv
$ sort -k 2n data.csv
$ sort -k 2r data.csv
$ sort -t ',' -k 2 data.csv
```

Option	Description
`-k <field>`	Sort by the specified field (e.g., `-k 2` sorts by the second field).
`-k <start>,<end>`	Sort by a range of fields, e.g., `-k 2,3` sorts by the second and third fields.
`-k <field>n`	Sort numerically on the specified field, e.g., `-k 2n` sorts the second field numerically.
`-k <field>r`	Sort in reverse order on the specified field, e.g., `-k 2r` sorts the second field in reverse.

Searching for Text in ASCII Files

If you are looking for text within a file, use the `grep` command.

```
$ grep <pattern> <file>
```

secret.txt

site: github.com
user: Mohamed
pass: ABC123

Command	Usage	Example
`$ grep keyword file`	Search for the a keyword (case-sensitive) in a file.	`$ grep user secret.txt` `$ grep o secret.txt`
`$ grep -v keyword file`	Invert the match, displaying non-matching lines.	`$ grep -v o secret.txt`
`$ grep -i keyowrd file`	Ignoring case (case-insensitive).	`$ grep -i abc secret.txt`
`$ grep -c keyword file`	Count the number of matching lines. Only outputs the count, not the matching lines.	`$ grep -c o secret.txt` `$ grep -ci USER secret.txt`
`$ grep -n keyword file`	Count the number of matching lines. Displays the matching lines.	`$ grep -n o secret.txt` `$ grep -ni USER secret.txt`

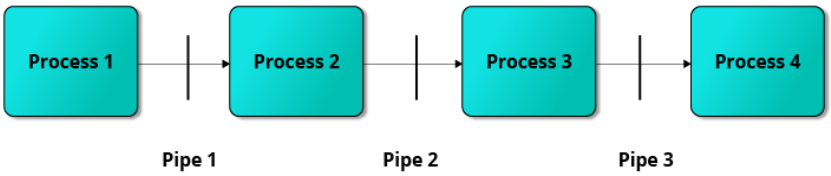
Pipes

The pipe (|) means take the standard output from the preceding command and pass it as the standard input to the following command. You can pipe the output of one command or program into another as its input.

```
$ command1 | command2 | command3 | ...
```

The above represents what we often call a pipeline and allows Linux to combine the actions of several commands into one.

Furthermore, there is no need to save output in (temporary) files between the stages in the pipeline, which saves disk space and reduces reading and writing from disk, which often constitutes the slowest bottleneck in getting something done.




```
$ ls -1 /etc | wc -l
$ ls -lah | sort
$ ls | grep -i FILE
$ compgen -c | wc -l

$ cat file.txt | grep Mohamed | uniq
$ cat /etc/passwd | less
$ cat /etc/passwd | cut -d: -f1 | sort

$ du -h | sort -h
$ du -ah /var/log | sort -rh | head -10

$ ps aux | grep httpd
$ dmesg | tail -10       (diagnostic messages)
$ dmesg | grep -i "eth"
```